Appendix: Artifact Description/Artifact Evaluation

Artifact Description (AD)

I. Overview of Contributions and Artifacts

A. Paper's Main Contributions

- C_1 We characterize the latency, bandwidth, and jitter sensitivity of AI communication workloads at scales up to 4,096 GPUs.
- C_2 We introduce a compositional linear-programming method with signature-based reuse for practical large-scale analysis.
- C_3 We use the results to derive network performance envelopes and identify useful provisioning points.

B. Computational Artifacts

The evaluation uses two artifacts:

- A_1 Code, input data, scripts, and results: https : / / github . com / tommasobo / SC26 _ ProvisioningAINetworks, branch artifact_freeze.
- A_2 Public execution traces: http://storage2.spcl.ethz.ch/traces/ai/.

Artifact	Contributions	Paper elements
A_1	C_1, C_2, C_3	Figs. 1, 3–10
A_2	C_1, C_2	Table II, Figs. 3–6

The requested badges are Artifacts Available, Artifacts Evaluated-Functional, and Results Reproduced. A persistent DOI will be added at the artifact-freeze stage.

II. Artifact Identification

A. Computational Artifact A_1

Relation To Contributions

Artifact A_1 contains the analysis implementation, plotting data, reproduction scripts, verification manifests, and final plots. The quick workflow generates Figures 1 and 3–10. Figure 2 is a method diagram. The full workflow additionally runs the analysis and validation stages for Figures 3–6.

Expected Results

The quick workflow should produce eight PDF files under figures/. The full workflow should also produce result CSV files, logs, and task summaries in the selected scratch directory. The generated curves should follow the paper results. Detailed numerical comparisons are provided in docs/REPRODUCTION_REPORT.md.

The artifact assumes that execution traces have already been collected. It does not launch model training or trace collection.

Expected Reproduction Time (in Minutes)

Allow 15–30 minutes for installation. The quick workflow takes about one minute. Allow up to eight hours for the full workflow, depending on the machine, solver license availability, and worker count. Most individual analysis tasks should be allowed up to one hour. The optional Grok 4k latency and bandwidth analysis should be allowed approximately 3 to 5 days.

Artifact Setup (incl. Inputs)

Hardware: The quick workflow needs one CPU core, 2 GB of RAM, and about 100 MB of free space. For the full workflow, we recommend at least 4 CPU cores, 32 GB of RAM, and a scratch filesystem. No GPU is required. The optional Grok 4k analysis should use a large-memory node with at least 512 GB of RAM and sufficient scratch space.

Software: Python 3.10 or newer is required. Python packages are listed in requirements.txt; full verification also uses requirements-dev.txt. The full analysis requires Gurobi and a valid license. The LogGOPSim example additionally uses g++, gengetopt, and re2c. Slurm is optional.

Datasets / Inputs: The repository contains the numerical and processed inputs needed by the default workflows. Large trace and intermediate files should be kept on a scratch filesystem. Public traces are available from http://storage2.spcl.ethz.ch/traces/ai/. Input identities are recorded under local_artifact/manifests/.

Installation and Deployment:

```
git clone https://github.com/tommasobo/\
SC26_ProvisioningAINetworks.git
cd SC26_ProvisioningAINetworks
git checkout artifact_freeze
python3.11 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
```

For the full workflow, also install requirements-dev.txt and confirm that Gurobi can obtain a license.

Artifact Execution

Run the quick workflow locally:

```
./reproduce_quick.sh
```

Run the full workflow with a scratch directory:

```
./reproduce_full.sh \
  --scratch /scratch/$USER/provisioning_artifact \
  --workers 4
```

On Alps, submit the analysis through Slurm:

```
./reproduce_full.sh --slurm \
  --account <account> --partition normal \
```

```
--scratch /iopsstor/scratch/cscs/$USER/artifact \
--workers 4
```

The optional Grok 4k analysis is enabled only with –expensive_run and an explicit N1024 input directory. Before enabling it, confirm that the selected partition provides at least 512 GB of RAM and a multi-day wall-time limit.

Artifact Analysis (incl. Outputs)

The quick workflow writes the final PDFs under figures/. The full workflow writes one summary per task together with result CSV files and logs in the selected scratch directory. Compare the produced files with the values in docs/REPRODUCTION_REPORT.md. The visual comparison is in docs/comparison/SC26_paper_vs_artifact.pdf.

B. Computational Artifact A_2

Relation To Contributions

Artifact A_2 contains the execution traces used to build communication graphs and exercise the analysis pipeline for Figures 3–6.

Expected Results

A selected trace should download with the recorded hash and pass through the trace-processing and simulation stages. The repository includes the input identities and reference results used by the final workflow.

Expected Reproduction Time (in Minutes)

Allow up to one hour for a selected trace download and replay. Processing a larger trace set can take several hours.

Artifact Setup (incl. Inputs)

Hardware: Use one CPU node with at least 32 GB of RAM and a scratch filesystem.

Software: The trace workflow uses curl, SQLite, the repository's Python tools, and LogGOPSim. NSYS is needed only when exporting a new report.

Datasets / Inputs: The public traces are available at http://storage2.spcl.ethz.ch/traces/ai/. Filenames and SHA-256 hashes used by this artifact are recorded under local_artifact/manifests/.

Installation and Deployment: Install Artifact A_1, select one trace recorded in the manifests, and download it to scratch. Do not store large trace files in the repository or the home directory.

Artifact Execution

Follow the trace-processing commands in docs/provenance/. The workflow converts the selected input into a communication schedule, replays it with LogGOPSim, and writes a result CSV.

Artifact Analysis (incl. Outputs)

Record the downloaded file's SHA-256 hash and the generated result. These values can be compared with the committed manifest and reference output.

Artifact Evaluation (AE)

A. Computational Artifact A_1

Artifact Setup (incl. Inputs)

Clone branch artifact_freeze, create the Python environment, and install requirements.txt. For full validation, install requirements-dev.txt, verify the Gurobi license, and choose a scratch directory. On a cluster, use a compute allocation or the Slurm launcher.

Artifact Execution

First run:

```
./reproduce_quick.sh
```

Confirm that Figures 1 and 3–10 are generated. Then run either the local full command or the Slurm command from the AD section. Do not add –expensive_run for the standard evaluation. The full workflow runs the Figure 3–6 analysis tasks and compares their outputs with the provided reference values.

Artifact Analysis (incl. Outputs)

Confirm that the eight expected PDF files exist under figures/. For a full run, inspect summary.json, the task summaries, and the result CSV files in scratch. Use docs/REPRODUCTION_REPORT.md for the expected numerical values.

B. Computational Artifact A_2

Artifact Setup (incl. Inputs)

Install Artifact A_1 and download one trace listed in the committed manifests to scratch. Verify its SHA-256 hash before processing it.

Artifact Execution

Use the documented trace workflow to produce a communication schedule, replay output, and result CSV. A single moderate trace is sufficient to check the pipeline.

Artifact Analysis (incl. Outputs)

Compare the input hash and replay result with the committed manifest and reference output.